

# exercise-3-recsys-2024

March 26, 2024

## 1 User-user Collaborative Filtering

---

RecSys Exercise 3, March 27, 2023

### 1.1 Overview

During this exercise you will implement a user-user collaborative filtering algorithm in Python.

*Please upload in Moodle this notebook (executable and in PDF) with the code and answers to the posted questions.*

### 1.2 Instructions

Before you start just to remind the intuitive algorithm of the user-user collaborative filtering:

1. Select neighbors - compare the current user's ratings against every other user's ratings by computing similarity (Pearson correlation).
2. Identify the most similar users to a particular target user - top N neighbors by similarity (correlation values: 1, the most similar user to  $-1$ , the least similar user).
3. Score items from neighbors' ratings (create predictions for the target user based on the ratings of the top most similar users). In other words, predict the rating the current user would give to every item he has not yet rated.
4. Recommend to the target user the items with highest predicted rating - top N recommendations.

## 2 Task 1: User-User collaborative filtering without normalization

For this task please use the dataset named `movie_ratings.csv`. The dataset is a 25 users x 100 movies matrix of ratings.

The dataset contains a significant number of missing values - do not replace these with zeros, they are correctly missing (those are movies which users hadn't rate). Moreover some additional functions that will help in the development of the task have been implemented-

```
[77]: from google.colab import drive
      drive.mount('/content/drive')
```

Drive already mounted at `/content/drive`; to attempt to forcibly remount, call `drive.mount("/content/drive", force_remount=True)`.

```

[78]: #Exercise 3: User-User Filtering
#Authors: Ariton Verush

import numpy as np
import pandas as pd
import math

#SupportFunctions

#This function receives the correlation values and a list of ratings, if the
    ↳rating is different than Nan
#the we multiply the value of the correlations by the rating otherwise no. The
    ↳sum of all the operations is returned.
def sumproduct(correlations, ratings):
    sum=0
    for i in range(len(list(correlations))):
        if not math.isnan(ratings[i]):
            sum=sum+(correlations[i]*ratings[i])
    return sum

#This function receives the correlation values and a list of ratings, if the
    ↳rating is different than Nan
#the we sum up the value of the correlations, otherwise no. The value of the
    ↳sum is returned
def sumif(correlations, ratings):
    sum=0
    for i in range(len(list(correlations))):
        if not math.isnan(ratings[i]):
            sum=sum+correlations[i]
    return sum

#Implementation of the Not Normalized score rating function
def prediction_not_normalized(correlations,ratings):
    if sumif(correlations.values.reshape(5), ratings)>0:
        return sumproduct(correlations.values.reshape(5), ratings) /
    ↳sumif(correlations.values.reshape(5), ratings)
    return np.nan

#*****MAIN
    ↳PROGRAM*****
# read data
movie_ratings = pd.read_csv('/content/movie-ratings.csv', header=0)
#copying dataset for task 2
movie_ratings_weighted=movie_ratings.copy()
movie_ratings

```

[78]:

|    | Movie                                             | Pedro | Pablo | Arianna \ |
|----|---------------------------------------------------|-------|-------|-----------|
| 0  | Star Wars: Episode IV - A New Hope (1977)         | NaN   | 4.5   | 5.0       |
| 1  | Finding Nemo (2003)                               | NaN   | 5.0   | 5.0       |
| 2  | Forrest Gump (1994)                               | NaN   | 5.0   | 4.5       |
| 3  | American Beauty (1999)                            | NaN   | 4.0   | NaN       |
| 4  | Pirates of the Caribbean: The Curse of the Bla... | 4.0   | 5.0   | 3.0       |
| .. | ...                                               | ...   | ...   | ...       |
| 95 | The Incredibles (2004)                            | 3.5   | 5.0   | 3.5       |
| 96 | Beauty and the Beast (1991)                       | 3.0   | 5.0   | NaN       |
| 97 | X-Men (2000)                                      | NaN   | 4.5   | NaN       |
| 98 | X2: X-Men United (2003)                           | 3.5   | 4.0   | NaN       |
| 99 | True Lies (1994)                                  | NaN   | NaN   | NaN       |

|    | Marcos | Arielle | Gio | Sofia | Moreno | Prisca | ... | Vanessa | Ginger | Allan \ |
|----|--------|---------|-----|-------|--------|--------|-----|---------|--------|---------|
| 0  | 4.5    | 4.0     | 4.0 | NaN   | 5.0    | 4.0    | ... | 4.0     | NaN    | 4.5     |
| 1  | NaN    | 4.0     | 4.0 | 4.5   | 4.5    | 4.0    | ... | 4.0     | NaN    | 3.5     |
| 2  | 5.0    | 4.5     | 4.5 | NaN   | 5.0    | 4.5    | ... | 4.0     | 5.0    | 3.5     |
| 3  | NaN    | NaN     | NaN | 4.5   | 2.0    | 3.5    | ... | 4.0     | NaN    | 3.5     |
| 4  | 4.5    | 4.0     | 2.5 | NaN   | 5.0    | 3.0    | ... | 3.0     | 1.5    | 4.0     |
| .. | ...    | ...     | ... | ...   | ...    | ...    | ... | ...     | ...    | ...     |
| 95 | NaN    | 3.0     | 3.5 | 4.5   | 5.0    | 3.5    | ... | 3.5     | 0.5    | 5.0     |
| 96 | NaN    | 4.0     | 3.5 | NaN   | 2.0    | 4.0    | ... | 4.0     | NaN    | 4.0     |
| 97 | NaN    | 4.0     | NaN | 4.5   | 5.0    | 4.0    | ... | 3.5     | NaN    | 3.0     |
| 98 | NaN    | 3.5     | NaN | 4.5   | 4.5    | 4.0    | ... | 3.5     | NaN    | 2.0     |
| 99 | NaN    | 3.0     | 3.5 | 4.0   | 1.5    | 3.0    | ... | NaN     | NaN    | 4.0     |

|    | Betty | Denisse | Ian | Ash | Misty | Brock | Tai |
|----|-------|---------|-----|-----|-------|-------|-----|
| 0  | 4.0   | 3.5     | NaN | NaN | NaN   | NaN   | NaN |
| 1  | 4.0   | 2.0     | 3.5 | NaN | NaN   | NaN   | 3.5 |
| 2  | 4.5   | 4.5     | 4.0 | 3.5 | 4.5   | 3.5   | 3.5 |
| 3  | 4.5   | 3.5     | 4.0 | NaN | 3.5   | NaN   | NaN |
| 4  | 4.0   | 2.5     | 3.5 | NaN | 5.0   | NaN   | 3.5 |
| .. | ...   | ...     | ... | ... | ...   | ...   | ... |
| 95 | 3.5   | 3.0     | NaN | NaN | 4.5   | NaN   | 3.5 |
| 96 | 2.0   | 2.5     | NaN | NaN | 4.0   | NaN   | NaN |
| 97 | 4.0   | 3.5     | NaN | NaN | NaN   | 3.0   | 4.0 |
| 98 | NaN   | 3.0     | 4.0 | NaN | 3.5   | NaN   | 4.0 |
| 99 | NaN   | 1.5     | 4.0 | NaN | NaN   | NaN   | NaN |

[100 rows x 26 columns]

[ ]:

A user-by-user correlations matrix has been calculated using the Pearson correlation model. Note that all correlation values are between -1 and 1 and that the diagonal is all 1's (since they are self-correlations).

```
[79]: #Calculating the pearson correlation among users
user_by_user_corr_matrix=movie_ratings.corr(method='pearson')
user_by_user_corr_matrix
```

<ipython-input-79-55363ce7d315>:2: FutureWarning: The default value of numeric\_only in DataFrame.corr is deprecated. In a future version, it will default to False. Select only valid columns or specify the value of numeric\_only to silence this warning.

```
user_by_user_corr_matrix=movie_ratings.corr(method='pearson')
```

```
[79]:
```

|         | Pedro     | Pablo    | Arianna   | Marcos    | Arielle   | Gio       | Sofia     | \ |
|---------|-----------|----------|-----------|-----------|-----------|-----------|-----------|---|
| Pedro   | 1.000000  | 0.402980 | -0.142206 | 0.517620  | 0.300200  | 0.480537  | -0.312412 |   |
| Pablo   | 0.402980  | 1.000000 | 0.118979  | 0.057916  | 0.341734  | 0.241377  | 0.131398  |   |
| Arianna | -0.142206 | 0.118979 | 1.000000  | -0.317063 | 0.294558  | 0.468333  | 0.092037  |   |
| Marcos  | 0.517620  | 0.057916 | -0.317063 | 1.000000  | -0.060913 | -0.008066 | 0.462910  |   |
| Arielle | 0.300200  | 0.341734 | 0.294558  | -0.060913 | 1.000000  | 0.282497  | 0.400275  |   |
| Gio     | 0.480537  | 0.241377 | 0.468333  | -0.008066 | 0.282497  | 1.000000  | 0.171151  |   |
| Sofia   | -0.312412 | 0.131398 | 0.092037  | 0.462910  | 0.400275  | 0.171151  | 1.000000  |   |
| Moreno  | 0.383348  | 0.206695 | -0.045854 | 0.214760  | 0.264249  | 0.072927  | 0.065015  |   |
| Prisca  | 0.092775  | 0.360056 | 0.367568  | 0.169907  | 0.125193  | 0.387133  | 0.095623  |   |
| Tue     | 0.098191  | 0.033642 | -0.035394 | 0.119350  | -0.333602 | 0.146158  | -0.292501 |   |
| Paula   | -0.041734 | 0.138548 | 0.011316  | 0.282756  | -0.066576 | 0.219929  | -0.038900 |   |
| Letty   | 0.264425  | 0.152948 | -0.231660 | -0.005326 | -0.093801 | -0.005316 | -0.364324 |   |
| James   | 0.261268  | 0.148882 | 0.148431  | -0.087747 | 0.310104  | 0.323499  | 0.126899  |   |
| Osito   | 0.464610  | 0.562449 | 0.267029  | 0.241567  | -0.003878 | 0.539066  | -0.051320 |   |
| Eva     | 0.022308  | 0.414438 | 0.304139  | 0.116532  | 0.113581  | 0.181276  | 0.227130  |   |
| Vanessa | -0.191988 | 0.488607 | 0.373226  | -0.201275 | 0.174085  | 0.347470  | 0.016406  |   |
| Ginger  | 0.493008  | 0.328120 | 0.470972  | 0.228341  | 0.297977  | 0.399436  | -0.240764 |   |
| Allan   | 0.360644  | 0.422236 | 0.069956  | 0.238700  | 0.476683  | 0.207314  | -0.115254 |   |
| Betty   | 0.551089  | 0.226635 | -0.054762 | 0.259660  | 0.293868  | 0.311363  | 0.247693  |   |
| Denisse | 0.002544  | 0.305803 | 0.133812  | 0.247097  | 0.438992  | 0.276306  | 0.166913  |   |
| Ian     | 0.116653  | 0.037769 | 0.015169  | 0.149247  | -0.162818 | 0.079698  | 0.146011  |   |
| Ash     | -0.429183 | 0.240728 | -0.273096 | -0.361466 | -0.295966 | 0.212991  | 0.009685  |   |
| Misty   | 0.394371  | 0.411676 | 0.082528  | 0.474974  | 0.054518  | 0.165608  | -0.451625 |   |
| Brock   | -0.304422 | 0.189234 | 0.667168  | -0.262073 | 0.464110  | 0.162314  | 0.193660  |   |
| Tai     | 0.245048  | 0.390067 | 0.119162  | 0.166999  | 0.379856  | 0.279677  | 0.113266  |   |

  

|         | Moreno    | Prisca   | Tue       | ... | Vanessa   | Ginger    | Allan     | \ |
|---------|-----------|----------|-----------|-----|-----------|-----------|-----------|---|
| Pedro   | 0.383348  | 0.092775 | 0.098191  | ... | -0.191988 | 0.493008  | 0.360644  |   |
| Pablo   | 0.206695  | 0.360056 | 0.033642  | ... | 0.488607  | 0.328120  | 0.422236  |   |
| Arianna | -0.045854 | 0.367568 | -0.035394 | ... | 0.373226  | 0.470972  | 0.069956  |   |
| Marcos  | 0.214760  | 0.169907 | 0.119350  | ... | -0.201275 | 0.228341  | 0.238700  |   |
| Arielle | 0.264249  | 0.125193 | -0.333602 | ... | 0.174085  | 0.297977  | 0.476683  |   |
| Gio     | 0.072927  | 0.387133 | 0.146158  | ... | 0.347470  | 0.399436  | 0.207314  |   |
| Sofia   | 0.065015  | 0.095623 | -0.292501 | ... | 0.016406  | -0.240764 | -0.115254 |   |
| Moreno  | 1.000000  | 0.028529 | -0.073252 | ... | 0.049132  | -0.009041 | 0.203613  |   |
| Prisca  | 0.028529  | 1.000000 | 0.210879  | ... | 0.475711  | 0.306957  | 0.136343  |   |

|         |           |           |           |     |           |           |           |
|---------|-----------|-----------|-----------|-----|-----------|-----------|-----------|
| Tue     | -0.073252 | 0.210879  | 1.000000  | ... | -0.040606 | 0.155045  | -0.204164 |
| Paula   | 0.312573  | 0.243283  | 0.022907  | ... | 0.079571  | -0.165628 | 0.053306  |
| Letty   | 0.053024  | -0.086061 | 0.252529  | ... | 0.072993  | 0.161882  | -0.000311 |
| James   | 0.143347  | 0.058365  | -0.221789 | ... | 0.101784  | -0.140953 | 0.150476  |
| Osito   | -0.118085 | 0.475495  | 0.258866  | ... | 0.326774  | 0.291476  | 0.372676  |
| Eva     | 0.100841  | 0.201734  | -0.024337 | ... | 0.251660  | 0.046822  | 0.218575  |
| Vanessa | 0.049132  | 0.475711  | -0.040606 | ... | 1.000000  | 0.086665  | 0.158739  |
| Ginger  | -0.009041 | 0.306957  | 0.155045  | ... | 0.086665  | 1.000000  | 0.149165  |
| Allan   | 0.203613  | 0.136343  | -0.204164 | ... | 0.158739  | 0.149165  | 1.000000  |
| Betty   | 0.033301  | 0.301750  | 0.263654  | ... | -0.016164 | 0.372177  | 0.276883  |
| Denisse | 0.137982  | 0.143414  | 0.167198  | ... | 0.256537  | 0.198086  | 0.158002  |
| Ian     | 0.070602  | 0.056100  | -0.084592 | ... | -0.055137 | 0.270928  | 0.035825  |
| Ash     | 0.109452  | 0.179908  | 0.315712  | ... | 0.503247  | -0.393376 | -0.345495 |
| Misty   | -0.083562 | 0.284648  | 0.085673  | ... | 0.100277  | 0.455274  | 0.449025  |
| Brock   | -0.089317 | 0.170757  | -0.109892 | ... | 0.423225  | 0.039050  | 0.289410  |
| Tai     | 0.229219  | 0.193131  | -0.279385 | ... | 0.222458  | 0.374264  | 0.169239  |

|         | Betty     | Denisse   | Ian       | Ash       | Misty     | Brock     | Tai       |
|---------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|
| Pedro   | 0.551089  | 0.002544  | 0.116653  | -0.429183 | 0.394371  | -0.304422 | 0.245048  |
| Pablo   | 0.226635  | 0.305803  | 0.037769  | 0.240728  | 0.411676  | 0.189234  | 0.390067  |
| Arianna | -0.054762 | 0.133812  | 0.015169  | -0.273096 | 0.082528  | 0.667168  | 0.119162  |
| Marcos  | 0.259660  | 0.247097  | 0.149247  | -0.361466 | 0.474974  | -0.262073 | 0.166999  |
| Arielle | 0.293868  | 0.438992  | -0.162818 | -0.295966 | 0.054518  | 0.464110  | 0.379856  |
| Gio     | 0.311363  | 0.276306  | 0.079698  | 0.212991  | 0.165608  | 0.162314  | 0.279677  |
| Sofia   | 0.247693  | 0.166913  | 0.146011  | 0.009685  | -0.451625 | 0.193660  | 0.113266  |
| Moreno  | 0.033301  | 0.137982  | 0.070602  | 0.109452  | -0.083562 | -0.089317 | 0.229219  |
| Prisca  | 0.301750  | 0.143414  | 0.056100  | 0.179908  | 0.284648  | 0.170757  | 0.193131  |
| Tue     | 0.263654  | 0.167198  | -0.084592 | 0.315712  | 0.085673  | -0.109892 | -0.279385 |
| Paula   | 0.007810  | -0.244637 | -0.030709 | -0.070660 | 0.268595  | -0.143503 | 0.013284  |
| Letty   | -0.077598 | 0.039389  | -0.156091 | 0.408592  | 0.179652  | 0.280402  | 0.040328  |
| James   | 0.024572  | -0.031130 | -0.133768 | 0.142067  | 0.015140  | 0.181210  | -0.005935 |
| Osito   | 0.525990  | 0.123380  | 0.178088  | 0.088600  | 0.668516  | 0.179680  | 0.155869  |
| Eva     | 0.150431  | 0.280392  | 0.038378  | 0.262520  | 0.064179  | -0.023439 | 0.257864  |
| Vanessa | -0.016164 | 0.256537  | -0.055137 | 0.503247  | 0.100277  | 0.423225  | 0.222458  |
| Ginger  | 0.372177  | 0.198086  | 0.270928  | -0.393376 | 0.455274  | 0.039050  | 0.374264  |
| Allan   | 0.276883  | 0.158002  | 0.035825  | -0.345495 | 0.449025  | 0.289410  | 0.169239  |
| Betty   | 1.000000  | 0.403809  | 0.028521  | 0.107821  | 0.428055  | 0.407044  | 0.278868  |
| Denisse | 0.403809  | 1.000000  | -0.068421 | 0.173797  | 0.105761  | 0.472361  | 0.257462  |
| Ian     | 0.028521  | -0.068421 | 1.000000  | -0.346386 | -0.004638 | 0.143672  | 0.074476  |
| Ash     | 0.107821  | 0.173797  | -0.346386 | 1.000000  | -0.339845 | 0.165960  | 0.156341  |
| Misty   | 0.428055  | 0.105761  | -0.004638 | -0.339845 | 1.000000  | 0.542192  | 0.435520  |
| Brock   | 0.407044  | 0.472361  | 0.143672  | 0.165960  | 0.542192  | 1.000000  | 0.080403  |
| Tai     | 0.278868  | 0.257462  | 0.074476  | 0.156341  | 0.435520  | 0.080403  | 1.000000  |

[25 rows x 25 columns]

The top 5 closest neighbors (the users with the 5 largest, positive correlations) for user **Sofia** have

been identified: *Marcos* (corr: 0.46291), *Arielle* (corr: 0.400275), *Betty* (corr: 0.247693), *Eva* (corr: 0.22713), and *Brock* (corr: 0.19366). Don't forget to exclude the target user (corr: 1.0) from your possible selections.

The line of code used to this end is:

```
[80]: #The Top5 similar users for user Sofia, the highest the correlation value, the
      ↪more similar. Observe that the dataframe
      #has been sliced from the index 1, since in the index 0 the value will be 1.00
      ↪(self-correlation)
      #To check the similar users of other users just change the label, e.g.,
      ↪user_by_user_corr_matrix.loc['Arielle']

corr_top5=(user_by_user_corr_matrix.loc['Sofia'].sort_values(ascending=False)[1:
      ↪6]).to_frame().T
corr_top5
```

```
[80]:      Marcos  Arielle  Betty  Eva  Brock
Sofia  0.46291  0.400275  0.247693  0.22713  0.19366
```

### Top 5 closest neighbors of user Arielle

Write down the top 5 closest neighbors for the user Arielle, you may use the code above

```
[81]: #Write your code here

Arielle_corr_top5=(user_by_user_corr_matrix.loc['Arielle'].
      ↪sort_values(ascending=False)[1:6]).to_frame().T
corr_top5
Arielle_corr_top5
```

```
[81]:      Allan  Brock  Denisse  Sofia  Tai
Arielle  0.476683  0.46411  0.438992  0.400275  0.379856
```

\*Answer: Arielle's top 5 neighbours are Allan(corr:0.476683), Brock(corr: 0.46411), Denisse(corr: 0.438992), Sofia(0.400275) and Tai(corr:0.379856), in the given order.

The lines of code that follow were written to get the prediction score for movies, based on the following not normalized scoring model:

Keep in mind that if a user hadn't rated an item, then the weights should not be considered in the computations.

Try to understand the code and proceed to the next point.

```
[82]: #This function receives the correlation values and a list of ratings, if the
      ↪rating is different than Nan
```

```

#the we multiply the value of the correlations by the rating otherwise no. The
↳sum of all the operations is returned.
def sumproduct(correlations, ratings):
    sum=0
    for i in range(len(list(correlations))):
        if not math.isnan(ratings[i]):
            sum=sum+(correlations[i]*ratings[i])
    return sum

#This function receives the correlation values and a list of ratings, if the
↳rating is different than Nan
#the we sum up the value of the correlations, otherwise no. The value of the
↳sum is returned
def sumif(correlations, ratings):
    sum=0
    for i in range(len(list(correlations))):
        if not math.isnan(ratings[i]):
            sum=sum+correlations[i]
    return sum

#Implementation of the Not Normalized score rating function
def prediction_not_normalized(correlations,ratings):
    if sumif(correlations.values.reshape(5), ratings)>0:
        return sumproduct(correlations.values.reshape(5), ratings) /
↳sumif(correlations.values.reshape(5), ratings)
    return np.nan

```

### Top 3 recommendations for user Arielle

In a real recommender system, we'd be excluding movies the user has already rated, but do not do this here. As an example, for the user Sofia, the correct submission would be:

1. Top Movie: Requiem for a dream, prediction: 5.000
2. 2nd Movie: The Matrix, prediction: 4856
3. 3rd Movie: Back to the future, prediction: 4.739

The code to obtain these results is presented in the following:

```

[83]: #This list is basically to select the rating of the top5 users
selection_labels= ['Movie']+ corr_top5.columns.tolist()
#print(selection_labels)
#Here we are using the previous list to select the ratings
rating_top5=movie_ratings.loc[:,selection_labels]
#print(rating_top5)

#Empty list to store the results of the prediction scores
prediction_results = []

```

```

#We iterate over the rows of the top5 similar users ratings
for index, row in rating_top5.iterrows():
    #Getting the rating values for the movies of each user of the top 5
    ratings_row=row[selection_labels[1:]].values
    #Computing the prediction values, using the not normalized model. We call
    ↪this function sending as parameters
    #the correlation values of the top 5 users and the ratings they have
    ↪assigned to the items
    pred_value=prediction_not_normalized(corr_top5,ratings_row)
    #List with the results of the prediction, we add a new result in each
    ↪iteration, one per each item
    prediction_results.append(pred_value)

#Adding a new column to our original DataFrame
movie_ratings['Prediction']=prediction_results

#Visualizing the items sorted from highest prediction score to lowest
#We should recommend the items with the highest prediction score
print(movie_ratings.sort_values(by='Prediction', ascending=False))

```

|    | Movie                           | Pedro | Pablo | Arianna | Marcos | Arielle | \ |
|----|---------------------------------|-------|-------|---------|--------|---------|---|
| 55 | Requiem for a Dream (2000)      | NaN   | 4.5   | NaN     | NaN    | NaN     |   |
| 50 | The Matrix (1999)               | 4.5   | 5.0   | 5.0     | 5.0    | 5.0     |   |
| 11 | Back to the Future (1985)       | 4.5   | 5.0   | 5.0     | NaN    | NaN     |   |
| 12 | Snatch (2000)                   | NaN   | NaN   | NaN     | 5.0    | NaN     |   |
| 21 | The Dark Knight (2008)          | 5.0   | 5.0   | NaN     | 5.0    | 5.0     |   |
| .. | ...                             | ...   | ...   | ...     | ...    | ...     |   |
| 32 | Fargo (1996)                    | NaN   | 5.0   | NaN     | NaN    | NaN     |   |
| 45 | Dances with Wolves (1990)       | NaN   | NaN   | NaN     | NaN    | NaN     |   |
| 62 | Almost Famous (2000)            | NaN   | 3.5   | NaN     | NaN    | NaN     |   |
| 92 | Clear and Present Danger (1994) | NaN   | NaN   | NaN     | NaN    | NaN     |   |
| 93 | Unbreakable (2000)              | NaN   | NaN   | NaN     | NaN    | NaN     |   |

|    | Gio | Sofia | Moreno | Prisca | ... | Ginger | Allan | Betty | Denisse | Ian | Ash | \ |
|----|-----|-------|--------|--------|-----|--------|-------|-------|---------|-----|-----|---|
| 55 | 3.0 | NaN   | 3.0    | NaN    | ... | 3.5    | NaN   | NaN   | 2.5     | 4.0 | 4.0 |   |
| 50 | 4.0 | 5.0   | 5.0    | 3.5    | ... | 5.0    | 4.0   | 4.5   | 4.5     | 4.0 | 3.0 |   |
| 11 | 4.5 | NaN   | 5.0    | 4.0    | ... | 5.0    | 3.5   | 4.5   | 4.5     | 4.5 | NaN |   |
| 12 | 3.5 | NaN   | 4.5    | 4.0    | ... | 3.5    | NaN   | 4.0   | 3.0     | 2.5 | NaN |   |
| 21 | 4.0 | NaN   | 5.0    | 3.5    | ... | 4.5    | 5.0   | 4.5   | 4.5     | 4.5 | 3.0 |   |
| .. | ... | ...   | ...    | ...    | ... | ...    | ...   | ...   | ...     | ... | ... |   |
| 32 | NaN | NaN   | 4.5    | 4.5    | ... | NaN    | NaN   | NaN   | 4.0     | NaN | 5.0 |   |
| 45 | 2.5 | 4.0   | 5.0    | 4.5    | ... | NaN    | 4.0   | NaN   | 2.5     | NaN | NaN |   |
| 62 | NaN | NaN   | 5.0    | 4.0    | ... | NaN    | NaN   | NaN   | 3.0     | 5.0 | NaN |   |
| 92 | NaN | NaN   | 2.5    | 3.5    | ... | NaN    | 3.0   | NaN   | 0.5     | NaN | NaN |   |
| 93 | NaN | NaN   | 3.5    | 3.5    | ... | NaN    | NaN   | NaN   | 3.0     | 3.0 | NaN |   |

Misty Brock Tai Prediction



```

55    NaN    NaN    NaN    5.000000
50    5.0    4.5    4.0    4.855924
11    NaN    NaN    2.0    4.739173
12    NaN    NaN    3.0    4.651432
21    4.5    5.0    3.5    4.622564
..    ...    ...    ...    ...
32    NaN    NaN    NaN    NaN
45    NaN    NaN    3.0    NaN
62    NaN    NaN    NaN    NaN
92    NaN    NaN    NaN    NaN
93    NaN    NaN    NaN    NaN

```

[100 rows x 27 columns]

[84]: *#Write the code to compute the recommendations for user Arielle her*

```

selection_labels= ['Movie']+ Arielle_corr_top5.columns.tolist()

rating_top5=movie_ratings.loc[:,selection_labels]
prediction_results = []

for index, row in rating_top5.iterrows():
    ratings_row=row[selection_labels[1:]].values
    pred_value=prediction_not_normalized(Arielle_corr_top5,ratings_row)
    prediction_results.append(pred_value)

movie_ratings['Prediction']=prediction_results

print(movie_ratings.sort_values(by='Prediction', ascending=False))

```

|    |                                                   | Movie | Pedro | Pablo | Arianna | \ |
|----|---------------------------------------------------|-------|-------|-------|---------|---|
| 77 | Star Wars: Episode V - The Empire Strikes Back... | NaN   | 4.5   | 5.0   |         |   |
| 21 | The Dark Knight (2008)                            | 5.0   | 5.0   | NaN   |         |   |
| 16 | The Lord of the Rings: The Return of the King ... | NaN   | 5.0   | NaN   |         |   |
| 8  | Memento (2000)                                    | NaN   | NaN   | 5.0   |         |   |
| 15 | The Lord of the Rings: The Two Towers (2002)      | NaN   | 5.0   | 4.5   |         |   |
| .. | ...                                               | ...   | ...   | ...   |         |   |
| 85 | Ace Ventura: Pet Detective (1994)                 | NaN   | 3.0   | 4.0   |         |   |
| 90 | Dumb & Dumber (1994)                              | NaN   | 1.0   | NaN   |         |   |
| 92 | Clear and Present Danger (1994)                   | NaN   | NaN   | NaN   |         |   |
| 58 | Harry Potter and the Chamber of Secrets (2002)    | 3.0   | 3.5   | 5.0   |         |   |
| 86 | Charlie's Angels (2000)                           | NaN   | 1.5   | NaN   |         |   |

  

|    | Marcos | Arielle | Gio | Sofia | Moreno | Prisca | ... | Ginger | Allan | Betty | \ |
|----|--------|---------|-----|-------|--------|--------|-----|--------|-------|-------|---|
| 77 | 4.0    | 4.5     | 4.0 | NaN   | 5.0    | 4.0    | ... | 1.0    | 5.0   | 4.5   |   |
| 21 | 5.0    | 5.0     | 4.0 | NaN   | 5.0    | 3.5    | ... | 4.5    | 5.0   | 4.5   |   |
| 16 | 4.0    | 5.0     | NaN | 5.0   | 5.0    | 4.0    | ... | NaN    | 5.0   | 5.0   |   |
| 8  | 5.0    | NaN     | 4.5 | NaN   | 3.0    | 4.5    | ... | 5.0    | 4.0   | 4.0   |   |

|    |     |     |     |     |     |     |     |     |     |     |
|----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 15 | 4.0 | 5.0 | NaN | 5.0 | 4.5 | 4.0 | ... | 1.0 | 4.0 | 4.5 |
| .. | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... |
| 85 | NaN | 2.5 | 3.5 | 5.0 | 1.0 | 3.5 | ... | 1.0 | 1.0 | NaN |
| 90 | NaN | 1.0 | 4.0 | 4.5 | 2.0 | 4.0 | ... | 0.5 | NaN | 4.0 |
| 92 | NaN | NaN | NaN | NaN | 2.5 | 3.5 | ... | NaN | 3.0 | NaN |
| 58 | 3.0 | 4.5 | 3.5 | NaN | 5.0 | 3.0 | ... | 0.5 | 3.0 | 2.0 |
| 86 | NaN | 3.0 | NaN | NaN | 1.0 | 2.5 | ... | NaN | 1.0 | 3.5 |

|    | Denisse | Ian | Ash | Misty | Brock | Tai | Prediction |
|----|---------|-----|-----|-------|-------|-----|------------|
| 77 | 4.5     | 4.5 | NaN | NaN   | NaN   | NaN | 4.760291   |
| 21 | 4.5     | 4.5 | 3.0 | 4.5   | 5.0   | 3.5 | 4.551454   |
| 16 | 4.5     | 4.5 | 2.5 | NaN   | 4.0   | 4.0 | 4.507637   |
| 8  | 4.5     | 5.0 | 4.0 | NaN   | 4.5   | 5.0 | 4.472487   |
| 15 | 4.0     | 4.5 | NaN | NaN   | 5.0   | 4.0 | 4.400194   |
| .. | ...     | ... | ... | ...   | ...   | ... | ...        |
| 85 | 0.5     | NaN | NaN | 3.0   | NaN   | 1.0 | 1.814717   |
| 90 | 0.5     | 4.0 | NaN | NaN   | NaN   | 0.5 | 1.813320   |
| 92 | 0.5     | NaN | NaN | NaN   | NaN   | NaN | 1.801453   |
| 58 | 0.5     | 2.5 | NaN | NaN   | NaN   | NaN | 1.801453   |
| 86 | 1.0     | 4.0 | NaN | NaN   | NaN   | 2.5 | 1.439808   |

[100 rows x 27 columns]

\*Answer: For Arielle, the correct order would be: 1. Star Wars: Episode V - The Empire Strikes Back, prediction: 4.760291 2. The Dark Knight, prediction: 4.551454 3. The Lord of the Rings: The Return of the King, prediction: 4.507637

### 3 Task 2: User-User collaborative filtering with normalized ratings

In this task you will perform similar steps as in Task 1, but this time you will normalize the ratings:

1. Write the necessary lines of code to compute the prediction score for each movie, by following the next model:

*Important:* Remember that there are items that have not being rated by certain users (NaN values), you must not consider them in the computations. Feel free to write your own functions to accomplish this point.

Add comments in the code to explain your solution.

```
[85]: #In this function, additionally we have the variables: length(size of the
      ↪ratings array), as well as the average_ratings. They are used
      #in calculating the average rating of all ratings, so we can use them next in
      ↪the normalized formula
      def sumproduct(correlations, ratings):
```

```

sum=0
length=0
average=0
for i in range(len(list(correlations))):
    if not math.isnan(ratings[i]):
        length=length+1
        average=average+ratings[i]

average=average/length
for i in range(len(list(correlations))):
    if not math.isnan(ratings[i]):
        sum=sum+(correlations[i]*(ratings[i]-average))
return sum

#This function receives the correlation values and a list of ratings, if the
→rating is different than Nan
#the we sum up the value of the correlations, otherwise no. The value of the
→sum is returned
def sumif(correlations, ratings):
    sum=0
    for i in range(len(list(correlations))):
        if not math.isnan(ratings[i]):
            sum=sum+correlations[i]
    return sum

#Implementation of the Normalized score rating function
def prediction_normalized(correlations,ratings):
    if sumif(correlations.values.reshape(5), ratings)>0:
        return sumproduct(correlations.values.reshape(5), ratings) /
        →sumif(correlations.values.reshape(5), ratings)
    return np.nan

```

```

[86]: from os import name
#This list is basically to select the rating of the top5 users, just like in
→task 1
selection_labels=['Movie']+corr_top5.columns.tolist()

#Here we are using the previous list to select the ratings
rating_top5=movie_ratings.loc[:,selection_labels]

#Empty list to store the results of the prediction scores
prediction_results = []

#Function to calculate the average ratings for a person
def average_ratings(movie_ratings, name):
    ratings=movie_ratings[name].values
    sum=0

```

```

length=0
average=0
for i in range(len(list(ratings))):
    if not math.isnan(ratings[i]):
        average=average+ratings[i]
        length=length+1
        average=average/length
    return average

#Like in task 1, we will first make calculations about Sofia
Sofia_average_rating=average_ratings(movie_ratings,"Sofia")

#We iterate over the rows of the top5 similar users ratings
for index, row in rating_top5.iterrows():
    #Getting the rating values for the movies of each user of the top 5
    ratings_row=row[selection_labels[1:]].values
    #Computing the prediction values, using the normalized model this time. We
    ↪ call this function sending as parameters
    #the correlation values of the top 5 users and the ratings they have
    ↪ assigned to the items
    pred_value=Sofia_average_rating+prediction_normalized(corr_top5,ratings_row)
    #List with the results of the prediction, we add a new result in each
    ↪ iteration, one per each item
    prediction_results.append(pred_value)

#Adding a new column to our original DataFrame
movie_ratings['Prediction']=prediction_results

#Visualizing the items sorted from highest prediction score to lowest
#We should recommend the items with the highest prediction score
print(movie_ratings.sort_values(by='Prediction', ascending=False))

```

|    | Movie                                             | Pedro | Pablo | Arianna | \ |
|----|---------------------------------------------------|-------|-------|---------|---|
| 4  | Pirates of the Caribbean: The Curse of the Bla... | 4.0   | 5.0   | 3.0     |   |
| 96 | Beauty and the Beast (1991)                       | 3.0   | 5.0   | NaN     |   |
| 85 | Ace Ventura: Pet Detective (1994)                 | NaN   | 3.0   | 4.0     |   |
| 2  | Forrest Gump (1994)                               | NaN   | 5.0   | 4.5     |   |
| 61 | V for Vendetta (2006)                             | NaN   | 4.0   | 5.0     |   |
| .. | ...                                               | ...   | ...   | ...     |   |
| 32 | Fargo (1996)                                      | NaN   | 5.0   | NaN     |   |
| 45 | Dances with Wolves (1990)                         | NaN   | NaN   | NaN     |   |
| 62 | Almost Famous (2000)                              | NaN   | 3.5   | NaN     |   |
| 92 | Clear and Present Danger (1994)                   | NaN   | NaN   | NaN     |   |
| 93 | Unbreakable (2000)                                | NaN   | NaN   | NaN     |   |

  

|   | Marcos | Arielle | Gio | Sofia | Moreno | Prisca | ... | Ginger | Allan | Betty | \ |
|---|--------|---------|-----|-------|--------|--------|-----|--------|-------|-------|---|
| 4 | 4.5    | 4.0     | 2.5 | NaN   | 5.0    | 3.0    | ... | 1.5    | 4.0   | 4.0   |   |

|    |     |     |     |     |     |     |     |     |     |     |
|----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 96 | NaN | 4.0 | 3.5 | NaN | 2.0 | 4.0 | ... | NaN | 4.0 | 2.0 |
| 85 | NaN | 2.5 | 3.5 | 5.0 | 1.0 | 3.5 | ... | 1.0 | 1.0 | NaN |
| 2  | 5.0 | 4.5 | 4.5 | NaN | 5.0 | 4.5 | ... | 5.0 | 3.5 | 4.5 |
| 61 | 4.0 | 5.0 | 4.0 | 4.5 | 2.0 | 3.5 | ... | 4.5 | NaN | NaN |
| .. | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... |
| 32 | NaN | NaN | NaN | NaN | 4.5 | 4.5 | ... | NaN | NaN | NaN |
| 45 | NaN | NaN | 2.5 | 4.0 | 5.0 | 4.5 | ... | NaN | 4.0 | NaN |
| 62 | NaN | NaN | NaN | NaN | 5.0 | 4.0 | ... | NaN | NaN | NaN |
| 92 | NaN | NaN | NaN | NaN | 2.5 | 3.5 | ... | NaN | 3.0 | NaN |
| 93 | NaN | NaN | NaN | NaN | 3.5 | 3.5 | ... | NaN | NaN | NaN |

|    | Denisse | Ian | Ash | Misty | Brock | Tai | Prediction |
|----|---------|-----|-----|-------|-------|-----|------------|
| 4  | 2.5     | 3.5 | NaN | 5.0   | NaN   | 3.5 | 4.788728   |
| 96 | 2.5     | NaN | NaN | 4.0   | NaN   | NaN | 4.735477   |
| 85 | 0.5     | NaN | NaN | 3.0   | NaN   | 1.0 | 4.706977   |
| 2  | 4.5     | 4.0 | 3.5 | 4.5   | 3.5   | 3.5 | 4.702242   |
| 61 | 4.5     | 3.5 | 2.5 | NaN   | 3.5   | 3.5 | 4.684436   |
| .. | ...     | ... | ... | ...   | ...   | ... | ...        |
| 32 | 4.0     | NaN | 5.0 | NaN   | NaN   | NaN | NaN        |
| 45 | 2.5     | NaN | NaN | NaN   | NaN   | 3.0 | NaN        |
| 62 | 3.0     | 5.0 | NaN | NaN   | NaN   | NaN | NaN        |
| 92 | 0.5     | NaN | NaN | NaN   | NaN   | NaN | NaN        |
| 93 | 3.0     | 3.0 | NaN | NaN   | NaN   | NaN | NaN        |

[100 rows x 27 columns]

2. Provide the top 3 recommendations for the user *Arielle* and the corresponding prediction values, in a similar manner Task 1.

```
[87]: #Write your code here

selection_labels=['Movie']+Arielle_corr_top5.columns.tolist()

rating_top5=movie_ratings.loc[:,selection_labels]
prediction_results = []

Arielle_average_rating=average_ratings(movie_ratings, "Arielle")

for index, row in rating_top5.iterrows():
    ratings_row=row[selection_labels[1:]].values
    ↵
    ↪pred_value=Arielle_average_rating+prediction_normalized(Arielle_corr_top5,ratings_row)
    prediction_results.append(pred_value)

movie_ratings['Prediction']=prediction_results

print(movie_ratings.sort_values(by='Prediction', ascending=False))
```

|    |                                                   | Movie                           | Pedro | Pablo | Arianna | \ |
|----|---------------------------------------------------|---------------------------------|-------|-------|---------|---|
| 91 |                                                   | The Lion King (1994)            | 4.0   | 5.0   | 5.0     |   |
| 20 |                                                   | Lost in Translation (2003)      | NaN   | 4.5   | NaN     |   |
| 11 |                                                   | Back to the Future (1985)       | 4.5   | 5.0   | 5.0     |   |
| 42 |                                                   | Spider-Man (2002)               | 4.0   | 3.0   | NaN     |   |
| 57 | Harry Potter and the Sorcerer's Stone (a.k.a. ... |                                 | NaN   | 4.0   | 5.0     |   |
| .. |                                                   | ...                             | ...   | ...   | ...     |   |
| 31 |                                                   | The Silence of the Lambs (1991) | NaN   | 5.0   | NaN     |   |
| 51 |                                                   | The Matrix Reloaded (2003)      | 4.0   | 4.0   | 4.5     |   |
| 66 |                                                   | Shrek 2 (2004)                  | NaN   | 4.0   | 5.0     |   |
| 24 |                                                   | Sin City (2005)                 | NaN   | 3.0   | 4.0     |   |
| 41 |                                                   | Fight Club (1999)               | NaN   | 5.0   | NaN     |   |

|    | Marcos | Arielle | Gio | Sofia | Moreno | Prisca | ... | Ginger | Allan | Betty | \ |
|----|--------|---------|-----|-------|--------|--------|-----|--------|-------|-------|---|
| 91 | 3.5    | 4.0     | 4.0 | NaN   | 5.0    | 4.5    | ... | 4.0    | 4.0   | NaN   |   |
| 20 | NaN    | NaN     | NaN | 4.0   | 5.0    | 3.5    | ... | NaN    | 4.0   | 4.5   |   |
| 11 | NaN    | NaN     | 4.5 | NaN   | 5.0    | 4.0    | ... | 5.0    | 3.5   | 4.5   |   |
| 42 | NaN    | 4.0     | 3.0 | NaN   | 5.0    | 3.0    | ... | 1.0    | 3.0   | NaN   |   |
| 57 | 4.0    | 4.0     | 3.0 | NaN   | 5.0    | 3.5    | ... | NaN    | 3.5   | 2.0   |   |
| .. | ...    | ...     | ... | ...   | ...    | ...    | ... | ...    | ...   | ...   |   |
| 31 | NaN    | NaN     | 4.5 | 4.5   | 4.5    | 4.5    | ... | 4.5    | 1.5   | NaN   |   |
| 51 | 5.0    | 2.5     | 2.5 | 5.0   | 5.0    | 4.0    | ... | 3.0    | 2.0   | 3.0   |   |
| 66 | 4.0    | 3.5     | 3.5 | 5.0   | 4.5    | 3.5    | ... | 1.5    | 1.5   | NaN   |   |
| 24 | NaN    | NaN     | NaN | NaN   | 1.5    | 3.5    | ... | 3.0    | 1.5   | 4.0   |   |
| 41 | 3.5    | 4.5     | 4.0 | NaN   | 4.5    | 4.0    | ... | 4.5    | NaN   | 5.0   |   |

|    | Denisse | Ian | Ash | Misty | Brock | Tai | Prediction |
|----|---------|-----|-----|-------|-------|-----|------------|
| 91 | 0.5     | 3.0 | 3.0 | 4.5   | NaN   | NaN | 4.072035   |
| 20 | 4.0     | NaN | NaN | NaN   | 4.5   | 1.5 | 4.067771   |
| 11 | 4.5     | 4.5 | NaN | NaN   | NaN   | 2.0 | 4.065710   |
| 42 | 3.5     | NaN | NaN | 3.5   | NaN   | 1.5 | 4.062951   |
| 57 | 0.5     | 5.0 | 3.0 | NaN   | NaN   | NaN | 4.061744   |
| .. | ...     | ... | ... | ...   | ...   | ... | ...        |
| 31 | 4.5     | 4.5 | NaN | 4.0   | 4.0   | 5.0 | 3.918410   |
| 51 | 2.5     | 5.0 | 3.0 | 4.0   | 1.5   | 3.5 | 3.913943   |
| 66 | 3.0     | NaN | NaN | 3.5   | NaN   | 3.5 | 3.912431   |
| 24 | 4.0     | 4.5 | NaN | NaN   | NaN   | 4.5 | 3.893408   |
| 41 | 2.0     | 4.5 | 5.0 | NaN   | NaN   | 5.0 | 3.891673   |

[100 rows x 27 columns]

\*Answer: With the normalized ratings, we get different top 3 movies for both Sofia and Arielle, as well as different prediction scores.

Sofia's top 3 movies:

-Pirates of the Caribbean: The Curse of the Black Pearl, prediction: 4.788728

-Beauty and the Beast, prediction: 4.735477

-Ace Ventura: Pet Detective, prediction: 4.706977

Arielle's top 3 movies:

-The Lion King, prediction: 4.072035

-Lost in Translation, prediction: 4.067771

-Back to the Future, prediction: 4.065710

3. Now that you have completed the tasks, please answer the following:

- Are there differences in the recommendations obtained in both approaches? > \*Answer: The difference is that the normalized approach is used to reduce the inconsistency in given data, and the non-normalized is just used to calculate fast and not so precise.
- In case of differences, which top 3 do you consider as 'best' recommendation for the user *Arielle*? > \*Answer: The results from the normalized approach.
- In which situation would you use a non-normalized approach and in which a normalized approach? > \*Answer: I would use a non-normalized approach if I would like to execute queries fast. On the other side, I would use a normalized approach when my data is important and I have the need for it to be consistent.
- Which other correlation models could you use to find user neighbors? > \*Answer: Cosine Similarity, Euclidean Distance, etc.